

Lab Assignment no. 2

PRACTICAL NO. 4

NAME: PREM VAIDYA

PRN:202501070120

BRANCH : ENTC

DIVISION :ET21

SUBJECT : EDS LAB

3.1.1.PANDAS – SERIES CREATION AND MANIPULATION

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers numbers
```

```
= list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers series
```

```
= pd.Series(numbers)
```

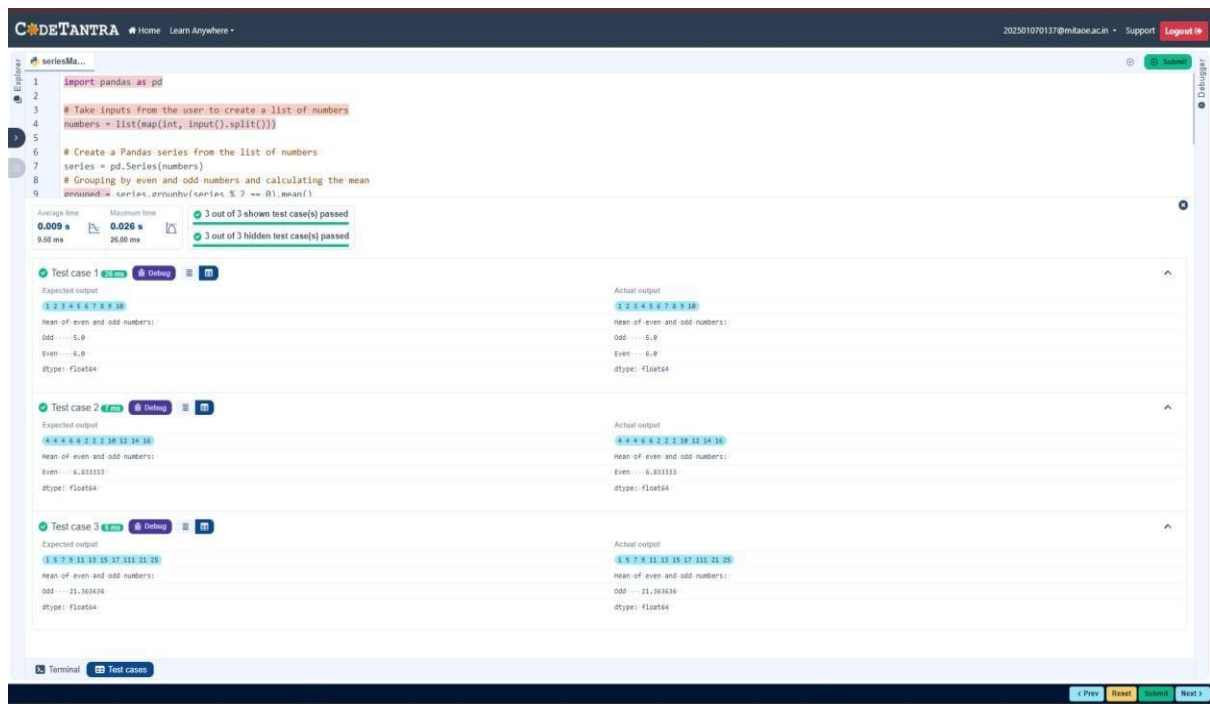
```
# Grouping by even and odd numbers and calculating the mean grouped
```

```
= series.groupby(series % 2 == 0).mean()
```

```
# Display the mean of even and odd numbers with labels
```

```
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
```

```
print("Mean of even and odd numbers:") print(grouped)
```



4.1.2.DICTIONARY TO DATAFRAME

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.

- **Display the DataFrame after modifying the row.**

Delete a row:

- **Take the row index to be deleted from the user.**
- **Remove the specified row.**
- **Display the DataFrame after deleting the row.**

Add a new column:

- **Add a column Gender with values taken from the user.**
- **Display the DataFrame after adding the new column.**

Modify a column:

- **Convert names to uppercase.**
- **Display the DataFrame after modifying the column.**

Delete a column:

- **Remove the Age column.**
- **Display the DataFrame after deleting the column.**

Code:

```
import pandas as pd
```

```
# Provided dictionary of lists data
```

```
= {
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
```

```
    'Age': [25, 30, 35], }
```

```
# Convert the dictionary to a DataFrame df
```

```
= pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:") print(df)
```

```
# Adding a new row new_name =
```

```
input("New name: ") new_age =
```

```
int(input("New age: ")) df.loc[len(df)]
```

```
= [new_name,new_age]
```

```
# Display the DataFrame after adding a new row print("After  
adding a row:\n",df)
```

```
# Modifying a row row_to_modify = int(input("Index  
of row to modify: ")) new_age_value = int(input("New  
age: ")) df.at[row_to_modify,"Age"] = new_age_value
```

```
# Display the DataFrame after modifying a row
```

```
print("After modifying a row:") print(df)
```

```
# Deleting a row row_to_delete = int(input("Index of  
row to delete: ")) df =
```

```
df.drop(index=row_to_delete).reset_index(drop=True)
```

```
# Display the DataFrame after deleting a row
```

```
print("After deleting a row:") print(df)
```

```
# Adding a new column genders = input("Enter genders  
separated by space: ").split() df["Gender"] = genders
```

```
# Display the DataFrame after adding a new column  
print("After adding a new column:") print(df)
```

```
# Modifying a column df["Name"] =  
df["Name"].str.upper() # Display the DataFrame  
after modifying a column print("After modifying a  
column:") print(df)
```

```
# Deleting a column df =  
df.drop(columns=["Age"])  
# Display the DataFrame after deleting a column print("After  
deleting a column:")  
print(df)
```

[Home](#)
[Learn Anywhere](#)

202501070137@mitaoe.ac.in - 5

datafram...

```

26 new_age_value = int(input("new age: "))
27 df.at[row_to_modify,"Age"] = new_age_value
28 # Display the DataFrame after modifying a row

```

Average time: 0.071 s

Maximum time: 0.085 s

71.50 ms

85.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1

Expected output

Original DataFrame:

```

.....Name Age
0.....Alice ..25
1.....Bob ..30
2.....Charlie ..35

```

New name: Susan

New age: 29

After adding a row:

```

.....Name Age
0.....Alice ..25
1.....Bob ..30
2.....Charlie ..35
3.....Susan ..29

```

Index of row to modify: 0

New age: 27

After modifying a row:

```

.....Name Age
0.....Alice ..27
1.....Bob ..30
2.....Charlie ..35
3.....Susan ..29

```

Index of row to delete: 3

After deleting a row:

```

.....Name Age
0.....Alice ..27
1.....Bob ..30
2.....Charlie ..35

```

Enter genders separated by space: F M M

After adding a new column:

```

.....Name Age Gender
0.....Alice ..27.....F
1.....Bob ..30.....M
2.....Charlie ..35.....M

```

After modifying a column:

```

.....Name Age Gender
0.....ALICE ..27.....F
1.....BOB ..30.....M
2.....CHARLIE ..35.....M

```

After deleting a column:

```

.....Name Gender
0.....ALICE.....F
1.....BOB.....M
2.....CHARLIE.....M

```

Actual output

Original DataFrame:

```

.....Name Age
0.....Alice ..25
1.....Bob ..30
2.....Charlie ..35

```

New name: Susan

New age: 29

After adding a row:

```

.....Name Age
0.....Alice ..25
1.....Bob ..30
2.....Charlie ..35
3.....Susan ..29

```

Index of row to modify: 0

New age: 27

After modifying a row:

```

.....Name Age
0.....Alice ..27
1.....Bob ..30
2.....Charlie ..35
3.....Susan ..29

```

Index of row to delete: 3

After deleting a row:

```

.....Name Age
0.....Alice ..27
1.....Bob ..30
2.....Charlie ..35

```

Enter genders separated by space: F M M

After adding a new column:

```

.....Name Age Gender
0.....Alice ..27.....F
1.....Bob ..30.....M
2.....Charlie ..35.....M

```

After modifying a column:

```

.....Name Age Gender
0.....ALICE ..27.....F
1.....BOB ..30.....M
2.....CHARLIE ..35.....M

```

After deleting a column:

```

.....Name Gender
0.....ALICE.....F
1.....BOB.....M
2.....CHARLIE.....M

```

4.1.3.STUDENT INFORMATION

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).

- **Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').**

Note:

Refer to the displayed test cases for better understanding.

Code:

```
import pandas as pd
```

```
# Read the text file into a DataFrame file = input() data = pd.read_csv(file,  
sep="\s+", header=None, names=["Name", "Age", "Grade"])
```

```
print("First five rows:") print(data.head())
```

```
average_age=round(data['Age'].mean(),2)
```

```
print(f"Average age: {average_age}")
```

```
threshold_grade='B'
```

```
filtered_data=data[data['Grade']<=threshold_grade]
```

```
print(f"Students with a grade up to {threshold_grade}")
```

```
print(filtered_data)
```


studentin...

Submit

Explorer

1

import pandas as pd

2

3

Read the text file into a DataFrame

4

file = input()

5

data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])

6

7

8

print("First five rows:")

9

print(data.head())

10

average_age=round(data['Age'].mean(),2)

11

print(f"Average age: {average_age}")

12

threshold_grade='B'

13

filtered_data=data[data['Grade']<=threshold_grade]

14

print(f"Students with a grade up to {threshold_grade}")

15

print(filtered_data)

Average time

0.049 s

49.00 ms

Maximum time

0.075 s

75.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 75 ms Debug

Expected output

Actual output

studentdata.txt

studentdata.txt

First five rows:

First five rows:

...

Name

...

Age

...

Grade

...

Name

...

Age

...

Grade

0 John 25 A

0 John 25 A

1 Alin 22 B

1 Alin 22 B

2 Emma 24 A

2 Emma 24 A

3 Mike 23 C

3 Mike 23 C

4 Sony 21 B

4 Sony 21 B

Average age: 23.29

Average age: 23.29

Students with a grade up to B

Students with a grade up to B

...

Name

...

Age

...

Grade

...

Name

...

Age

...

Grade

0 John 25 A

0 John 25 A

1 Alin 22 B

1 Alin 22 B

2 Emma 24 A

2 Emma 24 A

4 Sony 21 B

4 Sony 21 B

5 Amia 26 A

5 Amia 26 A

Terminal

Test cases

4.2.1.MONTH WITH THE HIGHEST TOTAL SALES

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.

- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name file_name
```

```
= input()
```

```
# Load the data df =
```

```
pd.read_csv(file_name)
```

```
df['Total_Sales']=df['Quantity']*df['Price']
```

```

df['Date']=pd.to_datetime(df['Date'])
df['Month']=df['Date'].dt.to_period('M')
monthly_sales=df.groupby('Month')['Total_
Sales'].sum() # Find the month with the
highest total sales best_month =
monthly_sales.idxmax() highest_sales =
monthly_sales.max()

print(f"Best month: {best_month}") print(f"Total
sales: ${highest_sales:.2f}")

```

The screenshot shows a code editor with a Python script and its test results. The script reads a CSV file, calculates monthly total sales, and identifies the best month and highest sales. The test results show that the script passed all test cases.

```

1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9
10 df['Total_Sales']=df['Quantity']*df['Price']
11 df['Date']=pd.to_datetime(df['Date'])
12 df['Month']=df['Date'].dt.to_period('M')
13 monthly_sales=df.groupby('Month')['Total_Sales'].sum()
14 # Find the month with the highest total sales
15 best_month = monthly_sales.idxmax()
16 highest_sales = monthly_sales.max()
17
18 print(f"Best month: {best_month}")
19 print(f"Total sales: ${highest_sales:.2f}")
20

```

Test Results:

- Average time: 0.033 s (32.67 ms)
- Maximum time: 0.067 s (67.00 ms)
- 1 out of 1 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1 (67 ms) - Debug

Expected output	Actual output
sales_data.csv	sales_data.csv
Best month: 2025-01	Best month: 2025-01
Total sales: \$1210.00	Total sales: \$1210.00

Terminal | Test cases

4.2.2.BEST SELLING PRODUCT

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name file_name
```

```
= input()
```

```
# Load the data df =
pd.read_csv(file_name)
df['Total_Sales']=df['Quantity']*df['Price']
product_sales=df.groupby('Product')['Quantity'].sum()

# Find the product with the highest total quantity sold best_product
=product_sales.idxmax() highest_quantity = product_sales.max()

# Display the result print(f"Best selling
product:          {best_product}")
print(f"Total      quantity      sold:
{highest_quantity}")
```

```

1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9
10 df['Total_Sales'] = df['Quantity'] * df['Price']
11 product_sales = df.groupby('Product')['Total_Sales'].sum()
12
13 # Find the product with the highest total quantity sold
14 best_product = product_sales.idxmax()
15 highest_quantity = product_sales.max()
16
17 # Display the result
18 print(f"Best selling product: {best_product}")
19 print(f"Total quantity sold: {highest_quantity}")
20

```

Average time: 0.026 s, Maximum time: 0.055 s, 1 out of 1 shown test case(s) passed, 2 out of 2 hidden test case(s) passed.

Test case 1: 55 ms, Debug, Expected output: sales_data.csv, Actual output: sales_data.csv, Best-selling product: Product A, Total quantity sold: 23.

4.2.3.CITY THAT SOLD THE MOST PRODUCT

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- **The CSV file contains the columns: Date, Product, Quantity, Price, and City.**
- **Group the data by City and calculate the total quantity of products sold for each city.**
- **Find the city that sold the most products (based on the total quantity sold).**

Sample Data:

<u>Date</u>	<u>Product</u>	<u>Quantity</u>	<u>Price</u>	<u>City</u>
<u>2025-01-01</u>	<u>Product A</u>	<u>5</u>	<u>20</u>	<u>New York</u>
<u>2025-01-01</u>	<u>Product B</u>	<u>3</u>	<u>15</u>	<u>Los Angeles</u>
<u>2025-01-02</u>	<u>Product A</u>	<u>7</u>	<u>20</u>	<u>New York</u>

<u>2025-01-02</u>	<u>Product C</u>	<u>4</u>	<u>30</u>	<u>Chicago</u>
<u>2025-01-03</u>	<u>Product B</u>	<u>2</u>	<u>15</u>	<u>Chicago</u>
<u>2025-01-03</u>	<u>Product A</u>	<u>8</u>	<u>20</u>	<u>Los Angeles</u>
<u>2025-01-04</u>	<u>Product C</u>	<u>6</u>	<u>30</u>	<u>New York</u>
<u>2025-01-04</u>	<u>Product B</u>	<u>5</u>	<u>15</u>	<u>Los Angeles</u>
<u>2025-01-05</u>	<u>Product A</u>	<u>3</u>	<u>20</u>	<u>Chicago</u>
<u>2025-01-05</u>	<u>Product C</u>	<u>10</u>	<u>30</u>	<u>Los Angeles</u>

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name file_name
```

```
= input()
```

```
# Load the data df =
```

```
pd.read_csv(file_name)
```

```
city_totals=df.groupby('City
```

```
')['Quantity'].sum()
```

```
best_city=city_totals.idxma
```

```
x()
```

```
# write the code..
```

```
# Display the result print(f"City sold the most
```

```
products: {best_city}")
```

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 city_totals=df.groupby('City')['Quantity'].sum()
10 best_city=city_totals.idxmax()
11 # write the code..
12
13 # Display the result
14 print(f"City sold the most products: {best_city}")
15
```

Average time: 0.024 s (23.67 ms) | Maximum time: 0.049 s (49.00 ms) | 1 out of 1 shown test case(s) passed | 2 out of 2 hidden test case(s) passed

Test case 1 (49 ms) | Debug | [Icons]

Expected output: sales_data.csv | Actual output: sales_data.csv

City sold the most products: Los Angeles | City sold the most products: Los Angeles

Terminal | Test cases

4.2.4.MOST FREQUENTLY SOLD PRODUCT PAIRS

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles

2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A • 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)
- Product A and Product C (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```

Code: import pandas as pd from
itertools import combinations
from collections import Counter

# Prompt user to input the file name file_name
= input()

# Read data from the specified CSV file df
= pd.read_csv(file_name)

product_pairs=[] for date,group in
df.groupby('Date'):
    products=group['Product'].unique()
    if len(products)>1:
        pairs=combinations(sorted(products),2)
    product_pairs.extend(pairs)

pair_counter=Counter(product_pairs)

max_freq=max(pair_counter.values()) most_frequent_pairs=[pair for pair ,count in
pair_counter.items() if count==max_freq]

for pair in most_frequent_pairs:    print(f"{pair[0]} and
{pair[1]}; {max_freq} times")

```

Explorer

frequentl...

Submit

```

1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 product_pairs=[]
12 for date,group in df.groupby('Date'):
13     products=group['Product'].unique()
14     if len(products)>1:
15         pairs=combinations(sorted(products),2)
16         product_pairs.extend(pairs)
17
18 pair_counter=Counter(product_pairs)
19
20 max_freq=max(pair_counter.values())
21 most_frequent_pairs=[pair for pair,count in pair_counter.items() if count==max_freq]
22
23 for pair in most_frequent_pairs:
24     print(f"{pair[0]} and {pair[1]}: {max_freq} times")
25
26 # Output the most frequent product pairs

```

Average time:

0.026 s

26.00 ms

Maximum time:

0.050 s

50.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

50 ms

Debug

Expected output

Actual output

sales_data.csv	sales_data.csv
Product A and Product B: 2 times	Product A and Product B: 2 times
Product A and Product C: 2 times	Product A and Product C: 2 times

Terminal

Test cases

4.2.5.TITANIC DATASET ANALYSIS AND DATA CLEANING

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.

2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
3	1	3	"Heikkinen, Miss. Laina"	female	26	0	0	STON/O2. 3101282	7.925		S
4	1	1	"Futrelle, Mrs. Jacques Heath (Lily May Peel)"	female	35	1	0	113803	53.1	C123	S
5	0	3	"Allen, Mr. William Henry"	male	35	0	0	373450	8.05		S
6	0	3	"Moran, Mr. James"	male		0	0	330877	8.4583		Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Code: import pandas as pd

import numpy as np

Load the Titanic dataset data = pd.read_csv('Titanic-Dataset.csv')

1. Display the first 5 rows of the dataset

print(data.head(5))

2. Display the last 5 rows of the dataset print(data.tail(5))

3. Get the shape of the dataset print(data.shape)

4. Get a summary of the dataset (info)

print(data.info())

5. Get basic statistics of the dataset print(data.describe())

6. Check for missing values

```
print(data.isnull().sum())
```

Explorer	titanicDat...	0	Submit
Average time	0.200 s	Maximum time	0.200 s
200.00 ms		200.00 ms	
Expected output	Actual output		
--- PassengerId Survived Pclass ... Fare Cabin Embarked	--- PassengerId Survived Pclass ... Fare Cabin Embarked		
0 1 0 3 7.2500 NaN S	0 1 0 3 7.2500 NaN S		
1 2 1 1 71.2833 C85 C	1 2 1 1 71.2833 C85 C		
2 3 1 3 7.9250 NaN S	2 3 1 3 7.9250 NaN S		
3 4 1 1 53.1000 C123 S	3 4 1 1 53.1000 C123 S		
4 5 0 3 8.0500 NaN S	4 5 0 3 8.0500 NaN S		
[5 rows x 12 columns]	[5 rows x 12 columns]		
--- PassengerId Survived Pclass ... Fare Cabin Embarked	--- PassengerId Survived Pclass ... Fare Cabin Embarked		
886 887 0 2 13.00 NaN S	886 887 0 2 13.00 NaN S		
887 888 1 1 30.00 B42 S	887 888 1 1 30.00 B42 S		
888 889 0 3 23.45 NaN S	888 889 0 3 23.45 NaN S		
889 890 1 1 30.00 C148 C	889 890 1 1 30.00 C148 C		
890 891 0 3 7.75 NaN Q	890 891 0 3 7.75 NaN Q		
[5 rows x 12 columns]	[5 rows x 12 columns]		
(891, 12)	(891, 12)		
<class 'pandas.core.frame.DataFrame'>	<class 'pandas.core.frame.DataFrame'>		
RangeIndex: 891 entries, 0 to 890	RangeIndex: 891 entries, 0 to 890		
Data columns (total 12 columns):	Data columns (total 12 columns):		
# Column Non-Null Count Dtype	# Column Non-Null Count Dtype		
0 PassengerId 891 non-null int64	0 PassengerId 891 non-null int64		
1 Survived 891 non-null int64	1 Survived 891 non-null int64		
2 Pclass 891 non-null int64	2 Pclass 891 non-null int64		
3 Name 891 non-null object	3 Name 891 non-null object		
4 Sex 891 non-null object	4 Sex 891 non-null object		
5 Age 714 non-null float64	5 Age 714 non-null float64		
6 SibSp 891 non-null int64	6 SibSp 891 non-null int64		
7 Parch 891 non-null int64	7 Parch 891 non-null int64		
8 Ticket 891 non-null object	8 Ticket 891 non-null object		
9 Fare 891 non-null float64	9 Fare 891 non-null float64		
10 Cabin 284 non-null object	10 Cabin 284 non-null object		
11 Embarked 889 non-null object	11 Embarked 889 non-null object		
dtypes: float64(2), int64(5), object(5)	dtypes: float64(2), int64(5), object(5)		
memory usage: 66.2+ KB	memory usage: 66.2+ KB		
None	None		
Terminal	Test cases		

PassengerId Survived Pclass ... SibSp Parch ...	PassengerId Survived Pclass ... SibSp Parch ...
Fare	Fare
count 891.000000 891.000000 891.000000 ... 891.000000 891.000000 891.000000 891.000000 891.000000 891.000000 891.000000 891.000000	count 891.000000 891.000000 891.000000 ... 891.000000 891.000000 891.000000 891.000000 891.000000 891.000000 891.000000 891.000000
mean 446.000000 0.383838 2.308642 ... 0.523000 0.381594 3	mean 446.000000 0.383838 2.308642 ... 0.523000 0.381594 3
std 257.353842 0.486592 0.836071 ... 1.102743 0.806057 4	std 257.353842 0.486592 0.836071 ... 1.102743 0.806057 4
min 1.000000 0.000000 1.000000 ... 0.000000 0.000000	min 1.000000 0.000000 1.000000 ... 0.000000 0.000000
25% 223.500000 0.000000 2.000000 ... 0.000000 0.000000	25% 223.500000 0.000000 2.000000 ... 0.000000 0.000000
75% 668.500000 1.000000 3.000000 ... 1.000000 0.000000 3	75% 668.500000 1.000000 3.000000 ... 1.000000 0.000000 3
max 891.000000 1.000000 3.000000 ... 8.000000 6.000000 51	max 891.000000 1.000000 3.000000 ... 8.000000 6.000000 51
[8 rows x 7 columns]	[8 rows x 7 columns]
PassengerId 0	PassengerId 0
Survived 0	Survived 0
Pclass 0	Pclass 0
Name 0	Name 0
Sex 0	Sex 0
Age 177	Age 177
SibSp 0	SibSp 0
Parch 0	Parch 0
Ticket 0	Ticket 0
Fare 0	Fare 0
Cabin 687	Cabin 687
Embarked 2	Embarked 2
dtype: int64	dtype: int64
Terminal	Test cases

4.2.6.TITANIC DATASET ANALYSIS AND DATA CLEANING – 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.

2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Code: import pandas as pd

import numpy as np

Load the Titanic dataset data =

pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch']

1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)

data['IsAlone']=np.where(data['FamilySize']>0,0,1)

2. Convert 'Sex' to numeric (male: 0, female: 1)

data['Sex']=data['Sex'].map({'male':0,'female':1})

3. One-hot encode the 'Embarked' column

data=pd.get_dummies(data,columns=['Embarked'],drop_first=True)

4. Get the mean age of passengers print(data['Age'].mean())

5. Get the median fare of passengers print(data['Fare'].median())

6. Get the number of passengers by cla print(data['Pclass'].value_counts()) # 7. Get the number of passengers by gender print(data['Sex'].value_counts())

8. Get the number of passengers by survival status print(data['Survived'].value_counts())

9. Calculate the survival rate total=len(data)

survivals=data['Survived'].sum()

print((survivals/total))

10. Calculate the survival rate by gender print(data.groupby('Sex')['Survived'].mean())

The screenshot shows a Jupyter Notebook window titled "titanicDat...". The code in the notebook is as follows:

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8
9 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
10 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
11
12 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14
15 # 3. One-hot encode the 'Embarked' column
16 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
17
18 # 4. Get the mean age of passengers
19 print(data['Age'].mean())
```

Below the code, the execution results are shown. The average time is 0.084 s (84.00 ms) and the maximum time is 0.084 s (84.00 ms). A message indicates "1 out of 1 shown test case(s) passed".

Test case 1 (4 ms) is shown with the following details:

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3...491	3...491
1...216	1...216
2...184	2...184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0...577	0...577
1...314	1...314
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0...549	0...549
1...342	1...342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0.3838383838383838	0.3838383838383838
Sex	Sex
0...0.188908	0...0.188908
1...0.742838	1...0.742838
Name: Survived, dtype: float64	Name: Survived, dtype: float64

At the bottom, there are tabs for "Terminal" and "Test cases".

4.2.7. TITANIC DATASET ANALYSIS AND DATA CLEANING – 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

The Titanic dataset contains columns as shown below,

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
 6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
 7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
 8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
 9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
 10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Code: import pandas as pd

import numpy as np

Load the Titanic dataset data = pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch'] data['IsAlone'] =

np.where(data['FamilySize'] > 0, 0, 1) data = pd.get_dummies(data,
 columns=['Embarked'], drop_first=True)

1. Calculate the survival rate by class print(data.groupby('Pclass')['Survived'].mean())

2. Calculate the survival rate by embarked location

print(data.groupby('Embarked_S')['Survived'].mean()) # 3. Calculate the survival rate by
 family size print(data.groupby('FamilySize')['Survived'].mean())

4. Calculate the survival rate by being alone

print(data.groupby('IsAlone')['Survived'].mean()) # 5. Get the average fare by class
 print(data.groupby('Pclass')['Fare'].mean())

6. Get the average age by class print(data.groupby('Pclass')['Age'].mean())

7. Get the average age by survival status `print(data.groupby('Survived')['Age'].mean())`

8. Get the average fare by survival status `print(data.groupby('Survived')['Fare'].mean())`

9. Get the number of survivors by class

`print(data[data['Survived']==1]['Pclass'].value_counts())`

10. Get the number of non-survivors by class

`print(data[data['Survived']==0]['Pclass'].value_counts())`

The screenshot shows a Jupyter Notebook interface with two code cells. The first cell contains the code `print(data[data['Survived']==1]['Pclass'].value_counts())` and the second cell contains `print(data[data['Survived']==0]['Pclass'].value_counts())`. Both cells show the output of the `value_counts()` method for the `Pclass` column, filtered by survival status. The first cell's output shows counts for survivors (Survived=1) and the second cell's output shows counts for non-survivors (Survived=0).

Pclass	count
1	14
2	17
3	57

Pclass	count
1	6
2	4
3	81

4.2.8. TITANIC DATASET ANALYSIS AND DATA CLEANING – 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age ≥ 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Code: import pandas as pd

import numpy as np

```
# Load the Titanic dataset data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

#1. Get the number of survivors by gender

```
print(data[data['Survived']==1] ['Sex'].value_counts())
```

#2. Get the number of non-survivors by gender

```
print(data[data['Survived']==0] ['Sex'].value_counts())
```

#3. Get the number of survivors by embarked location

```
print(data[data['Survived']==1] ['Embarked_S'].value_counts())
```

#4. Get the number of non-survivors by embarked location print(data[data['Survived']==0]
['Embarked_S'].value_counts())

```
print(data[data['Age']<18] ['Survived'].mean())
```

#6. Calculate the percentage of adults (Age >= 18) who survived

```
print(data[data['Age']>=18] ['Survived'].mean())
```

```
print(data[data['Survived']==1] ['Age'].median())
```

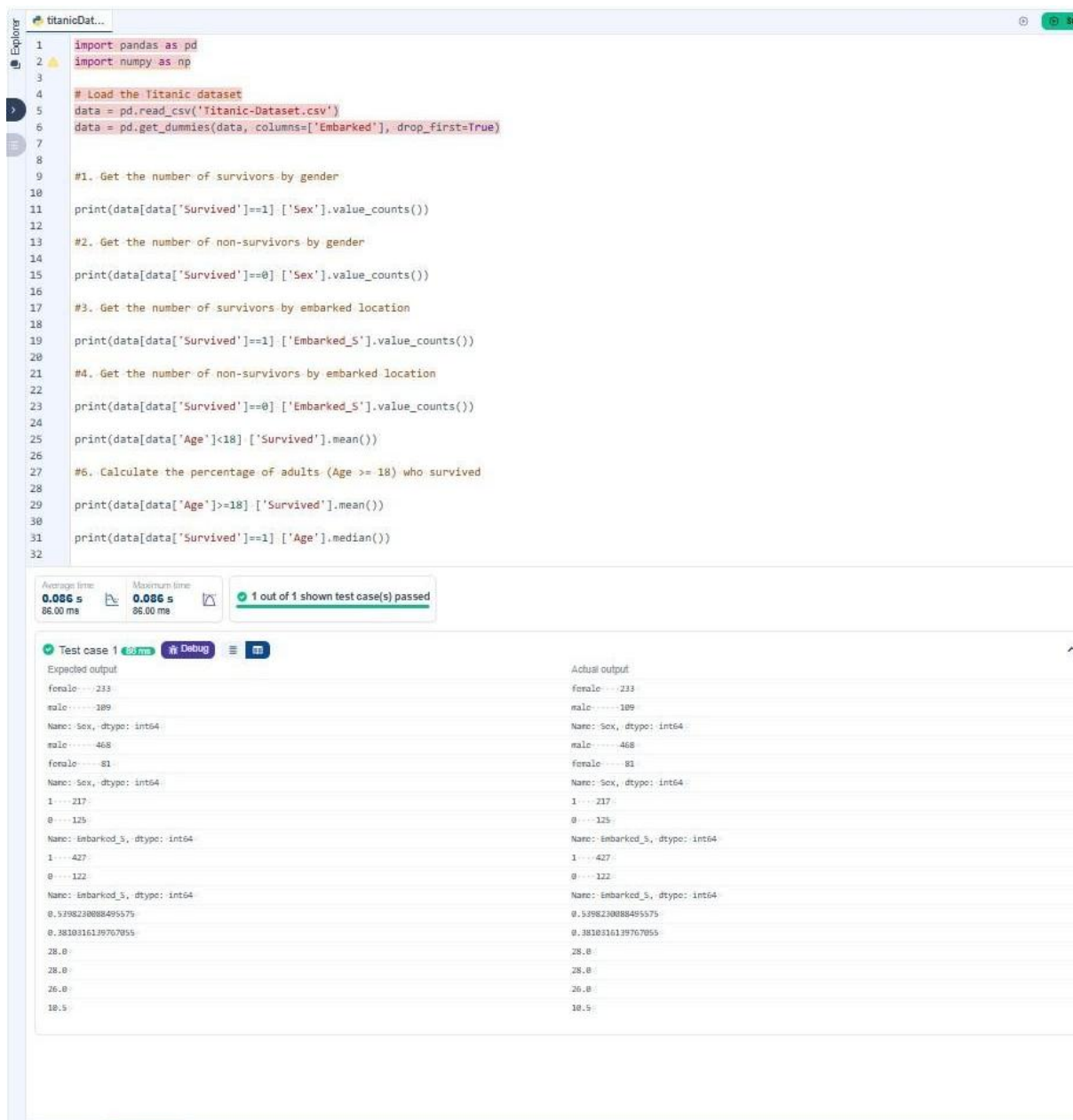
#8. Get the median age of non-survivors

```
print(data[data['Survived']==0] ['Age'].median())
```

```
print(data[data['Survived']==1] ['Fare'].median())
```

#10. Get the median fare of non-survivors

```
print(data[data['Survived']==0] ['Fare'].median())
```



The screenshot displays a Jupyter Notebook environment. The top section shows a code editor with Python code for loading the Titanic dataset and performing various analyses. The code includes imports for pandas and numpy, loading the dataset, creating dummy variables for embarked locations, and calculating statistics like value counts, means, and medians for survivors and non-survivors based on gender, embarked location, and age.

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9 #1. Get the number of survivors by gender
10 print(data[data['Survived']==1] ['Sex'].value_counts())
11
12 #2. Get the number of non-survivors by gender
13 print(data[data['Survived']==0] ['Sex'].value_counts())
14
15 #3. Get the number of survivors by embarked location
16 print(data[data['Survived']==1] ['Embarked_S'].value_counts())
17
18 #4. Get the number of non-survivors by embarked location
19 print(data[data['Survived']==0] ['Embarked_S'].value_counts())
20
21 print(data[data['Age']<18] ['Survived'].mean())
22
23 #6. Calculate the percentage of adults (Age >= 18) who survived
24 print(data[data['Age']>=18] ['Survived'].mean())
25
26 print(data[data['Survived']==1] ['Age'].median())
```

Below the code editor, the test results are shown. The average time for the test is 0.086 s, and the maximum time is 0.086 s. The test passed, with 1 out of 1 shown test case(s) passed.

Test case 1 (0.086 s) [Debug]

Expected output	Actual output
female: 233	female: 233
male: 389	male: 389
Name: Sex, dtype: int64	Name: Sex, dtype: int64
female: 468	female: 468
male: 81	male: 81
Name: Sex, dtype: int64	Name: Sex, dtype: int64
1: 217	1: 217
0: 125	0: 125
Name: Embarked_S, dtype: int64	Name: Embarked_S, dtype: int64
1: 427	1: 427
0: 122	0: 122
Name: Embarked_S, dtype: int64	Name: Embarked_S, dtype: int64
0.5798230088495575	0.5798230088495575
0.3810316139767055	0.3810316139767055
28.0	28.0
28.0	28.0
26.0	26.0
18.5	18.5

